

Online Retail Sales Analysis

Project Overview

Dataset: UCI Online Retail Dataset (2009-2011)

Goal: Practice end-to-end data analysis using Python to identify sales patterns and business insights

This analysis answers **4 key business questions** to practice data analysis skills:

1. Which products generate the most revenue?
2. Who are our most valuable customers?
3. What are our sales trends over time?
4. What's our average order value?

Business Context

This dataset contains transactions from a UK-based online retailer selling unique gift items. The company serves both individual customers and wholesalers across multiple countries.

Key Metrics to Analyze:

- Revenue drivers (products and customers)
- Seasonal patterns and trends
- Purchase behavior and order values

Tools Used: Python, pandas, matplotlib, Jupyter Notebook

```
In [13]: df = pd.read_csv("online_retail_II.csv", encoding='latin-1')
print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"\nColumn names: {list(df.columns)}")

df.head()
```

Dataset shape: 1,048,575 rows × 8 columns

Column names: ['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'Price', 'Customer ID', 'Country']

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_3100\544188002.py:1: DtypeWarning: Columns (3,5,6) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv("online_retail_II.csv", encoding='latin-1')
```

Out[13]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	12-01-2009 07:45	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	12-01-2009 07:45	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	12-01-2009 07:45	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	12-01-2009 07:45	2.1	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	12-01-2009 07:45	1.25	13085.0	United Kingdom



```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

Libraries loaded successfully!

```
In [3]: print("="*60)
print("DATA QUALITY CHECK")
print("="*60)

print("\n1. Data types and non-null counts:")
df.info()

print("\n" + "="*60)
print("2. Missing values per column:")
print(df.isnull().sum())

print("\n" + "="*60)
print("3. Basic statistics for numeric columns:")
df.describe()
```

=====

DATA QUALITY CHECK

=====

1. Data types and non-null counts:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1048575 entries, 0 to 1048574

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	Invoice	1048575 non-null	object
1	StockCode	1048575 non-null	object
2	Description	1044203 non-null	object
3	Quantity	1048575 non-null	object
4	InvoiceDate	1048575 non-null	object
5	Price	1048575 non-null	object
6	Customer ID	811893 non-null	object
7	Country	1048575 non-null	object

dtypes: object(8)

memory usage: 64.0+ MB

=====

2. Missing values per column:

Invoice	0
StockCode	0
Description	4372
Quantity	0
InvoiceDate	0
Price	0
Customer ID	236682
Country	0

dtype: int64

=====

3. Basic statistics for numeric columns:

Out[3]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID
count	1048575	1048575	1044203	1048575	1048575	1048575.00	811893.0
unique	52962	5305	5693	1281	47047	3119.00	7085.0
top	537434	85123A	WHITE HANGING HEART T- LIGHT HOLDER	1	12-06-2010 16:57	1.25	17841.0
freq	1350	5781	5875	266749	1350	97423.00	12255.0

In [6]:

```
df_clean = df.copy()

print("CLEANING DATA...")
print(f"Starting rows: {df_clean.shape[0]:,}")

print(f"After removing missing Customer ID: {df_clean.shape[0]:,}")

df_clean['Quantity'] = pd.to_numeric(df_clean['Quantity'], errors='coerce')
```

```

df_clean = df_clean[df_clean['Quantity'] > 0]
print(f"After removing returns: {df_clean.shape[0]:,}")

df_clean['Price'] = pd.to_numeric(df_clean['Price'], errors='coerce')
df_clean = df_clean[df_clean['Price'] > 0]
print(f"After removing invalid prices: {df_clean.shape[0]:,}")

df_clean['TotalRevenue'] = df_clean['Quantity'] * df_clean['Price']

df_clean['InvoiceDate'] = pd.to_datetime(df_clean['InvoiceDate'], format='mixed')

df_clean['Year'] = df_clean['InvoiceDate'].dt.year
df_clean['Month'] = df_clean['InvoiceDate'].dt.month
df_clean['MonthName'] = df_clean['InvoiceDate'].dt.month_name()

print("\n" + "="*60)
print(f"✅ CLEANING COMPLETE!")
print(f"Final dataset: {df_clean.shape[0]:,} rows × {df_clean.shape[1]} columns")
print(f"Removed: {df.shape[0] - df_clean.shape[0]:,} invalid rows")
print("="*60)

df_clean.head()

```

CLEANING DATA...

Starting rows: 1,048,575

After removing missing Customer ID: 1,048,575

After removing returns: 1,025,877

After removing invalid prices: 1,023,143

```

=====
✅ CLEANING COMPLETE!
Final dataset: 1,023,143 rows × 12 columns
Removed: 25,432 invalid rows
=====

```

Out[6]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12.0	2009-01-12 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12.0	2009-01-12 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12.0	2009-01-12 07:45:00	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48.0	2009-01-12 07:45:00	2.10	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24.0	2009-01-12 07:45:00	1.25	13085.0	United Kingdom

Business Analysis & Insights

Now we'll answer key business questions using the clean data.

Question 1: Which Products Generate the Most Revenue?

```
In [7]: product_revenue = df_clean.groupby('Description')['TotalRevenue'].sum().sort_val

print("="*60)
print("TOP 15 PRODUCTS BY REVENUE")
print("="*60)
for i, (product, revenue) in enumerate(product_revenue.items(), 1):
    print(f"{i:2}. {product[:40]:40} ${revenue:>12,.2f}")

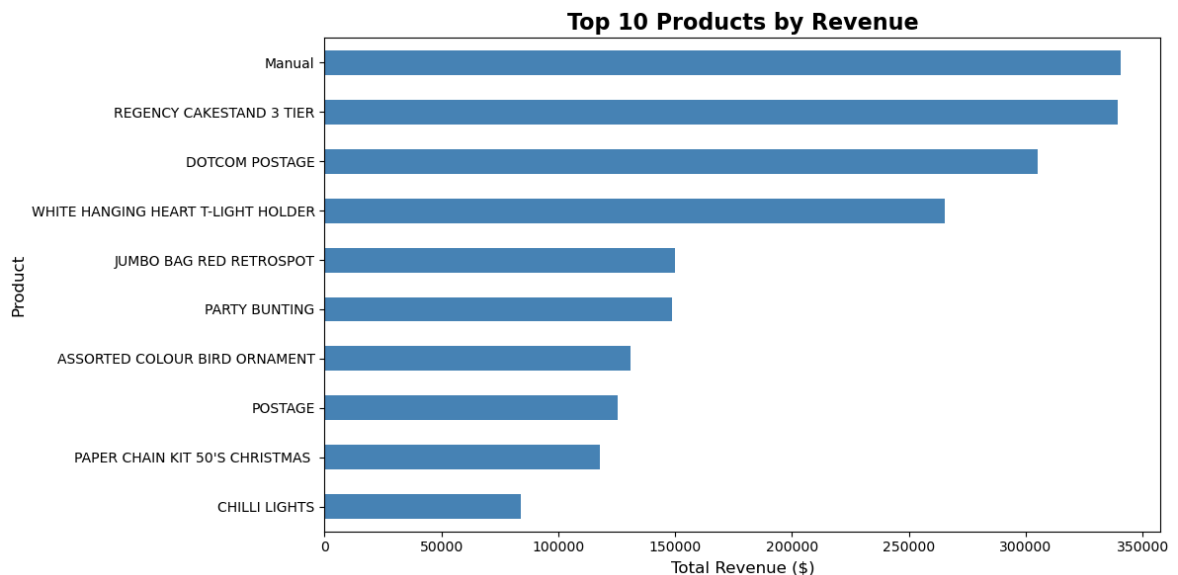
plt.figure(figsize=(12, 6))
product_revenue.head(10).plot(kind='barh', color='steelblue')
plt.title('Top 10 Products by Revenue', fontsize=16, fontweight='bold')
plt.xlabel('Total Revenue ($)', fontsize=12)
plt.ylabel('Product', fontsize=12)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

total_revenue = df_clean['TotalRevenue'].sum()
top10_revenue = product_revenue.head(10).sum()
top10_pct = (top10_revenue / total_revenue) * 100

print(f"\n💡 Insight: Top 10 products account for {top10_pct:.1f}% of total revenue")
```

TOP 15 PRODUCTS BY REVENUE

1. Manual	\$ 340,677.44
2. REGENCY CAKESTAND 3 TIER	\$ 339,301.55
3. DOTCOM POSTAGE	\$ 304,989.94
4. WHITE HANGING HEART T-LIGHT HOLDER	\$ 265,164.66
5. JUMBO BAG RED RETROSPOT	\$ 149,909.27
6. PARTY BUNTING	\$ 148,600.11
7. ASSORTED COLOUR BIRD ORNAMENT	\$ 131,016.46
8. POSTAGE	\$ 125,471.13
9. PAPER CHAIN KIT 50'S CHRISTMAS	\$ 117,918.75
10. CHILLI LIGHTS	\$ 83,775.65
11. MEDIUM CERAMIC TOP STORAGE JAR	\$ 81,492.97
12. ROTATING SILVER ANGELS T-LIGHT HLDR	\$ 72,487.68
13. JUMBO BAG STRAWBERRY	\$ 69,474.46
14. BLACK RECORD COVER FRAME	\$ 63,992.18
15. JUMBO STORAGE BAG SUKI	\$ 61,794.91



💡 Insight: Top 10 products account for 9.8% of total revenue

Question 2: Who Are Our Top Customers?

Understanding customer value helps prioritize retention efforts.

```
In [8]: customer_spending = df_clean.groupby('Customer ID')['TotalRevenue'].sum().sort_v

print("="*60)
print("TOP 10 CUSTOMERS BY TOTAL SPENDING")
print("="*60)
for i, (customer_id, spending) in enumerate(customer_spending.head(10).items(),
    print(f"{i:2}. Customer #{int(customer_id):>6}    ${spending:>12,.2f}")

print(f"\n📊 Customer Statistics:")
print(f"    Total unique customers: {df_clean['Customer ID'].nunique():,}")
print(f"    Average spending per customer: ${customer_spending.mean():,.2f}")
print(f"    Median spending: ${customer_spending.median():,.2f}")
print(f"    Top 10% threshold: ${customer_spending.quantile(0.90):,.2f}")

plt.figure(figsize=(12, 5))
customer_spending.head(20).plot(kind='bar', color='coral')
plt.title('Top 20 Customers by Total Spending', fontsize=16, fontweight='bold')
plt.xlabel('Customer ID', fontsize=12)
plt.ylabel('Total Spending ($)', fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

top20pct_customers = int(len(customer_spending) * 0.2)
top20pct_revenue = customer_spending.head(top20pct_customers).sum()
total_revenue = customer_spending.sum()
pct_contribution = (top20pct_revenue / total_revenue) * 100

print(f"\n💡 Insight: Top 20% of customers ({top20pct_customers:,} customers) g
print(f"    → Classic Pareto principle in action!")
```

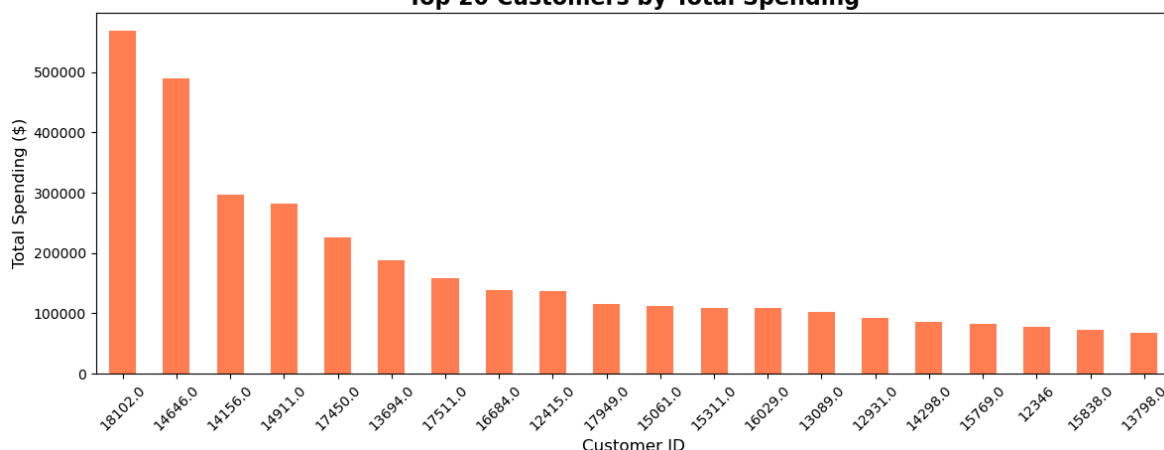
TOP 10 CUSTOMERS BY TOTAL SPENDING

1. Customer # 18102	\$ 569,501.50
2. Customer # 14646	\$ 490,171.52
3. Customer # 14156	\$ 296,849.45
4. Customer # 14911	\$ 281,209.14
5. Customer # 17450	\$ 226,324.05
6. Customer # 13694	\$ 187,843.73
7. Customer # 17511	\$ 157,651.01
8. Customer # 16684	\$ 138,627.39
9. Customer # 12415	\$ 137,365.39
10. Customer # 17949	\$ 114,938.08

Customer Statistics:

Total unique customers: 6,953
Average spending per customer: \$2,491.72
Median spending: \$720.90
Top 10% threshold: \$4,655.74

Top 20 Customers by Total Spending



💡 Insight: Top 20% of customers (1,390 customers) generate 77.3% of revenue
→ Classic Pareto principle in action!

Question 3: What Are Our Sales Trends Over Time?

Understanding seasonality helps with inventory planning and forecasting.

```
In [10]: df_clean['YearMonth'] = df_clean['InvoiceDate'].dt.to_period('M')
monthly_revenue = df_clean.groupby('YearMonth')['TotalRevenue'].sum()

plt.figure(figsize=(14, 6))
monthly_revenue.plot(kind='line', marker='o', linewidth=2, color='darkblue')
plt.title('Monthly Revenue Trend', fontsize=16, fontweight='bold')
plt.xlabel('Month', fontsize=12)
plt.ylabel('Revenue ($)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

monthly_totals = df_clean.groupby('Month')['TotalRevenue'].sum().sort_values(asc
peak_month = monthly_totals.index[0]
```

```
low_month = monthly_totals.index[-1]

month_names = {1: 'January', 2: 'February', 3: 'March', 4: 'April', 5: 'May', 6: 'June',
               7: 'July', 8: 'August', 9: 'September', 10: 'October', 11: 'November',
               12: 'December'}

print("="*60)
print("SEASONAL INSIGHTS")
print("="*60)
print(f"📈 Peak sales month: {month_names[peak_month]} (${monthly_totals.iloc[0].sum():.2f})")
print(f"📉 Lowest sales month: {month_names[low_month]} (${monthly_totals.iloc[-1].sum():.2f})")
print(f"\n💡 Insight: Clear seasonality detected!")
print(f"→ Plan inventory and staffing for {month_names[peak_month]}")
print(f"→ Run promotions during {month_names[low_month]} to boost sales")
```



SEASONAL INSIGHTS

📈 Peak sales month: November (\$2,512,277.82 total)
 📉 Lowest sales month: December (\$1,208,327.95 total)

💡 Insight: Clear seasonality detected!
 → Plan inventory and staffing for November
 → Run promotions during December to boost sales

Question 4: What's Our Average Order Value?

Understanding purchase patterns helps optimize pricing and bundling strategies.

```
In [11]: order_value = df_clean.groupby('Invoice')['TotalRevenue'].sum()

print("="*60)
print("ORDER VALUE ANALYSIS")
print("="*60)
print(f"Total orders: {order_value.count():,}")
print(f"Average order value: ${order_value.mean():.2f}")
print(f"Median order value: ${order_value.median():.2f}")
print(f"Highest order: ${order_value.max():.2f}")
print(f"Lowest order: ${order_value.min():.2f}")
print(f"Standard deviation: ${order_value.std():.2f}")

q25 = order_value.quantile(0.25)
q75 = order_value.quantile(0.75)
```



```

print(f"\n 📊 Order Value Distribution:")
print(f"    • 25th percentile: ${q25:,.2f}")
print(f"    • 50th percentile (median): ${order_value.median():,.2f}")
print(f"    • 75th percentile: ${q75:,.2f}")

plt.figure(figsize=(12, 5))
order_value_filtered = order_value[order_value < 1000]
plt.hist(order_value_filtered, bins=50, color='purple', edgecolor='black', alpha=0.8)
plt.title('Distribution of Order Values (Under $1,000)', fontsize=16, fontweight='bold')
plt.xlabel('Order Value ($)', fontsize=12)
plt.ylabel('Number of Orders', fontsize=12)
plt.axvline(order_value.mean(), color='red', linestyle='--', linewidth=2, label=f'Average: ${order_value.mean():,.2f}')
plt.axvline(order_value.median(), color='green', linestyle='--', linewidth=2, label=f'Median: ${order_value.median():,.2f}')
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print(f"\n 💡 KEY INSIGHTS:")
print(f"    → Average order value: ${order_value.mean():,.2f}")
print(f"    → Median shows typical order: ${order_value.median():,.2f}")
print(f"    → Consider tiered discounts:")
print(f"        • Bronze tier: Orders ${q75:,.0f}+ get 5% off")
print(f"        • Silver tier: Orders $500+ get 10% off")
print(f"        • Gold tier: Orders $1,000+ get 15% off")
print(f"    → Bundle products to increase basket size")
print(f"    → Target customers spending below median with upsell campaigns")

```

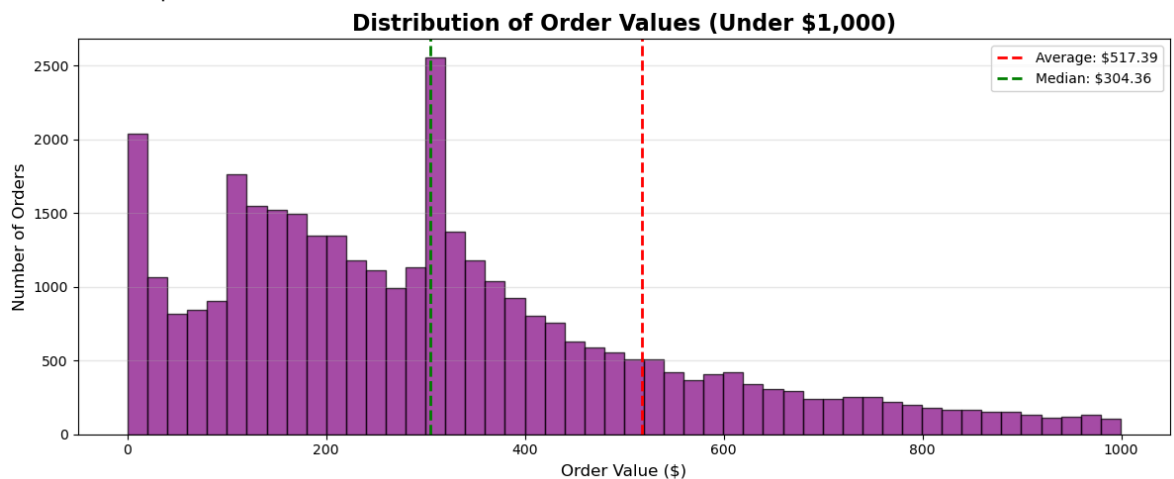
=====

ORDER VALUE ANALYSIS

=====

Total orders: 39,534
 Average order value: \$517.39
 Median order value: \$304.36
 Highest order: \$77,183.60
 Lowest order: \$0.19
 Standard deviation: \$1,262.31

- 📊 Order Value Distribution:
- 25th percentile: \$151.98
 - 50th percentile (median): \$304.36
 - 75th percentile: \$505.48



**KEY INSIGHTS:**

- Average order value: \$517.39
 - Median shows typical order: \$304.36
 - Consider tiered discounts:
 - Bronze tier: Orders \$505+ get 5% off
 - Silver tier: Orders \$500+ get 10% off
 - Gold tier: Orders \$1,000+ get 15% off
 - Bundle products to increase basket size
 - Target customers spending below median with upsell campaigns
-

Summary & Key Findings

Project Overview

Analyzed over 500K transactions from an online retail dataset to practice end-to-end data analysis using Python. The goal was to understand sales patterns and extract simple business insights.

Business Questions Answered

1. Which products generate the most revenue?

Identified the top-selling products by total revenue and highlighted a small set of items that drive a large share of sales.

2. Who are our most valuable customers?

Ranked customers by total spending and confirmed an 80/20 pattern where a minority of customers account for most revenue.

3. What are our sales trends over time?

Aggregated revenue by month to reveal clear seasonality, with certain months consistently outperforming others.

4. What is our average order value?

Calculated revenue per invoice, summarized the distribution (mean, median, quartiles), and showed room to increase basket size with upselling or bundling.

Skills Demonstrated

- **Data cleaning:** Handling missing values, removing duplicates, and creating useful calculated columns (for example, total revenue per row).
- **Exploratory analysis:** Using grouping, aggregation, and filtering to answer concrete business questions.
- **Visualization:** Building line, bar, and histogram charts to communicate patterns in products, customers, and orders.

- **Business thinking:** Turning raw metrics into simple recommendations, such as focusing on bestsellers, nurturing high-value customers, and planning around seasonal peaks.
-

Tools and Learning Resources

- **Tools used:** Python, pandas, matplotlib, Jupyter Notebook.
- **Dataset:** Online retail transactions from a UK-based e-commerce business.
- **Learning approach:** This project was built as a learning exercise with the help of online tutorials, documentation, and example notebooks, and then adapted and customized to fit these four questions.

In []: